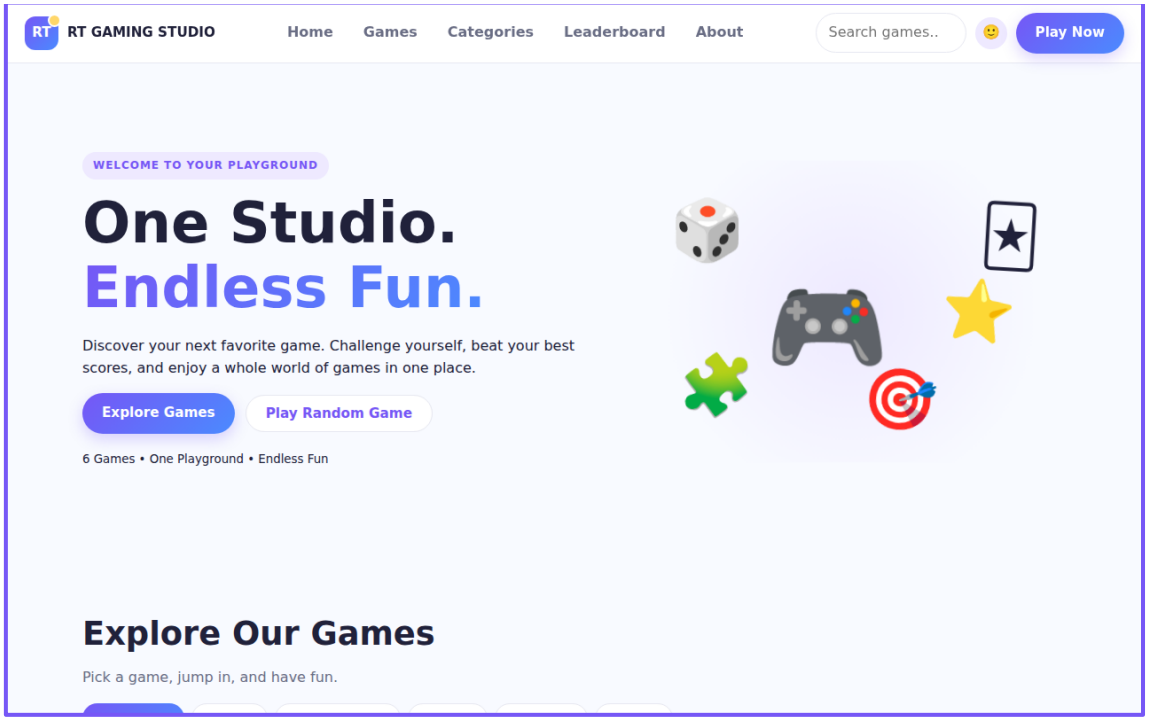
A PERSONAL PROJECT REPORT

RT GAMING STUDIO

A Web-Based Gaming Platform Using HTML, CSS and JavaScript



6	3	0	ES6+
PLAYABLE GAMES	CORE FILES	EXTERNAL DEPENDENCIES	VANILLA JAVASCRIPT

Project Title	RT Gaming Studio – A Web-Based Gaming Platform
Technologies	HTML5 · CSS3 · JavaScript (ES6+) · Canvas · LocalStorage
Project Type	Personal project (front-end only, no backend)
Developer ID	RT1504
Live Project	https://shahrishabh1513-jsk.github.io/RT_GAMING_STUDIO/
Source Code	https://github.com/shahrishabh1513-jsk/RT_GAMING_STUDIO

DESIGNED AND DEVELOPED BY

SHAH RISHABH ALPESHBHAI

CONTENTS

Section / Chapter	Title	Page No.
	Certificate	1
	Acknowledgement	2
	Abstract	3
CHAPTER 1	INTRODUCTION	4
1.1	Project Overview	4
1.2	Problem Statement	4
1.3	Objectives of the Project	5
1.4	Scope of the Project	5
CHAPTER 2	TOOLS AND TECHNOLOGIES	7
2.1	HTML5	7
2.2	CSS3	7
2.3	JavaScript	8
2.4	LocalStorage and HTML Canvas	8
CHAPTER 3	PROJECT DESIGN AND MODULES	9
3.1	System Architecture	9
3.2	Website Flow and Navigation	11
3.3	Loading Screen and Homepage	12
3.4	Game Library and Categories	12
3.5	Description of the Games	13
3.6	Scoreboard and Statistics	16
CHAPTER 4	SYSTEM IMPLEMENTATION	17
4.1	Project Folder Structure	17
4.2	Frontend Implementation	17
4.3	JavaScript Game Logic	18
4.4	Game Controls and Score Management	19
CHAPTER 5	SOFTWARE TESTING	20
5.1	Functional Testing	20
5.2	Game Testing	21
5.3	Responsive UI Testing	22
5.4	Test Case Summary	22
CHAPTER 6	SCREENSHOTS AND WALKTHROUGH	23
6.1	Loading Screen and Home Page	23
6.2	Game Library and Search	24
6.3	Gameplay Screenshots	25
6.4	Scoreboard and Statistics	28
CHAPTER 7	LEARNING OUTCOMES	30
7.1	Skills and Knowledge Gained	30
7.2	Challenges Faced and Solutions	30
7.3	Limitations and Future Scope	31
CHAPTER 8	CONCLUSION	32
8.1	Conclusion	32
8.2	Future Enhancements	32
CHAPTER 9	REFERENCES AND GLOSSARY	33
9.1	References	33
9.2	Glossary	33

CERTIFICATE

This is to certify that **SHAH RISHABH ALPESHBHAI** has successfully completed the personal project entitled **"RT GAMING STUDIO – A Web-Based Gaming Platform Using HTML, CSS and JavaScript"**.

The project was designed, developed, tested and documented by the candidate using HTML5, CSS3 and Vanilla JavaScript. The work presented in this report is original and has been carried out sincerely and to the best of the candidate's ability.

This report is submitted as a record of the candidate's personal project work.

Project Guide (Name & Signature)	Head of Department (Name & Signature)
Date: _____	Place: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported me during the planning, development and documentation of my personal project, **RT Gaming Studio**.

I am thankful to my project guide and the faculty members of my department for their valuable guidance, encouragement and constructive suggestions, which helped me to improve the quality of this work. Their feedback motivated me to focus on clean code, good design and a better user experience.

I also thank my family and friends for their constant support, patience and motivation throughout the project. I am grateful to the open web community and to the developers of freely available learning resources such as MDN Web Docs, which made it possible for me to learn and apply HTML, CSS and JavaScript effectively.

Finally, I thank everyone who directly or indirectly helped me in completing this project successfully.

SHAH RISHABH ALPESHBHAI

ABSTRACT

RT Gaming Studio is a web-based gaming platform that brings several mini-games together in a single, modern and responsive website. The project is developed using only HTML5, CSS3 and Vanilla JavaScript (ES6+), without any framework, library, backend or database. The complete website runs in a modern browser from three main files: index.html, style.css and script.js.

The platform provides a branded animated loading screen, a sticky navigation bar, an attractive hero section, a searchable and filterable game library, a “Game of the Day” section, a local leaderboard, an About section and a designed footer. Games open inside the same page in a shared game window controlled by a JavaScript game registry.

Six fully playable games are implemented with real game logic: Candy Match (a match-3 puzzle), Snakes and Ladders, SnakeBite, Tic-Tac-Toe with a minimax-based hard AI, Block Stack (a Tetris-style game) and Archery. Scores, wins, play counts and recently played games are saved in the browser using LocalStorage, with a safe fallback when storage is unavailable.

The interface follows a light colour theme, supports mouse, keyboard and touch input, adapts to desktop, tablet and mobile screens, and respects the reduced-motion preference of the user. The project demonstrates DOM manipulation, event handling, HTML Canvas drawing, game algorithms, state management and responsive web design.

Keywords: HTML5, CSS3, JavaScript, Canvas, LocalStorage, Web Games, Responsive Design

CHAPTER

1 INTRODUCTION

1.1 Project Overview

RT Gaming Studio is a browser-based gaming website designed as a single “playground” where users can discover, select and play multiple mini-games without installing anything. Instead of presenting only a landing page, the project integrates the website and the games: every Play Now button opens a real, working game inside the page.

The website is built with a clear separation between the site logic (navigation, search, filtering, statistics, game window) and the individual game logic. Each game is registered in a central game registry with its title, category, difficulty, instructions and a start function, so the shared game window can start, restart, pause and stop any game in the same way.

Main highlights of the project:

- Animated branded loading screen and a bright, light-themed homepage.
- Game library with category filters, live search and result counter.
- Six playable games with scoring, restart and instructions.
- Local leaderboard and statistics stored in LocalStorage.
- Responsive layout for desktop, tablet and mobile, with touch controls.
- Accessibility support: semantic HTML, keyboard focus styles, ARIA labels and reduced-motion support.

AT A GLANCE

6 playable games · 3 core files · 0 external dependencies · light theme · scores saved in the browser · works on desktop, tablet and mobile

1.2 Problem Statement

Casual players often have to visit many different websites or install several applications to play simple games. These sources may contain advertisements, require sign-up, load slowly or do not work well on small screens. Students who want to learn web

development also need practical projects that combine design, logic and interactivity rather than isolated examples.

There is a need for a simple, fast and attractive platform that:

- gathers different types of games in one place;
- works instantly in the browser with no installation or login;
- remembers the player's best scores on the same device;
- is usable on both computers and mobile phones; and
- is built with core web technologies so that the code is easy to read, learn and extend.

1.3 Objectives of the Project

- To design and develop a complete gaming website using only HTML5, CSS3 and Vanilla JavaScript.
- To create an original brand identity (RT Gaming Studio) with a consistent light theme.
- To implement a game library with category filters and search.
- To build playable games with genuine rules, scoring and restart functionality.
- To manage all games through a single game registry and shared game window.
- To store scores and statistics safely using LocalStorage.
- To support keyboard, mouse and touch interaction.
- To create a responsive, accessible and animated user interface.
- To properly clean up timers, animation frames and keyboard listeners when a game is closed.

1.4 Scope of the Project

Included in the scope:

- A front-end only website that runs by opening index.html in a modern browser.
- Home, Games, Categories, Leaderboard and About sections with smooth navigation.

- Six single-device games: Candy Match, Snakes and Ladders, SnakeBite, Tic-Tac-Toe, Block Stack and Archery.
- Local statistics: games played, wins, best scores, most-played game and recent activity.
- Light theme, responsive design and reduced-motion support.

Not included in the scope:

- User registration, login or any authentication system.
- Backend server, database or online multiplayer.
- A global leaderboard (the leaderboard is local to the browser and device).
- Paid APIs, external frameworks or third-party game engines.
- Sound effects (planned as a future enhancement).

CHAPTER

2 TOOLS AND TECHNOLOGIES

2.1 HTML5

HTML5 provides the structure of the website. Semantic elements such as header, nav, main, section, article and footer describe the meaning of each part of the page, which helps accessibility and maintainability. The project uses the following HTML5 features:

- **Semantic layout** – header with navigation, sections for hero, games, leaderboard and about, and a footer.
- **Canvas element** – used to draw SnakeBite, Block Stack and Archery.
- **Form controls** – search input, select lists (game mode and difficulty) and buttons.
- **Dialog behaviour** – a modal container with role="dialog" and aria-modal for the game window.
- **Accessibility attributes** – aria-label, aria-expanded, aria-live and alt text.

2.2 CSS3

CSS3 controls the appearance, layout and animation of the website. All styles are written in a single file, style.css, using CSS custom properties (variables) for the colour palette so that the theme stays consistent.

- **Flexbox and Grid** for the header, game cards, statistics cards, game boards and footer.
- **Gradients and shadows** used sparingly for the logo, buttons, progress bars and selected states.
- **Keyframe animations** for the loading logo, floating hero elements, modal opening and loading bar.
- **Transitions** for hover effects on cards, buttons and footer links.
- **Media queries** at 820 px and 480 px for tablet and mobile layouts.
- **prefers-reduced-motion** media query that disables non-essential animation.
- **:focus-visible** outlines for keyboard users.

2.3 JavaScript

JavaScript (ES6+) provides all the behaviour of the website and the games. The code is wrapped in an immediately invoked function expression with strict mode, which avoids global variable pollution. Important language features used are const and let, arrow functions, template literals, destructuring, spread syntax, Set, array methods (map, filter, some, every), async/await with Promises, classes of functions (one function per game) and optional chaining.

- **DOM manipulation** to render game cards, boards, statistics and the game window.
- **Event handling** for clicks, keyboard events, pointer events and input events.
- **Timers** (setInterval, setTimeout) and **requestAnimationFrame** for game loops.
- **Game algorithms** such as match detection, minimax search, collision detection and rotation of matrices.

2.4 LocalStorage and HTML Canvas

LocalStorage is a browser feature that stores small amounts of data as key–value text on the user's device. RT Gaming Studio stores one JSON object under the key "rtgs", containing play counts, best scores, total wins and recently played games. Every read and write is wrapped in try/catch; if LocalStorage is blocked or unavailable, the application keeps the data in memory so that the website never crashes.

HTML Canvas is a 2D drawing surface controlled by JavaScript. It is used where pixel-level drawing and fast redrawing are required: the grid and snake in SnakeBite, the board and tetrominoes in Block Stack, and the range, target, wind and arrows in Archery.

Development environment used in the project:

Item	Details
Languages	HTML5, CSS3, JavaScript (ES6+)
Code editor	Any text editor / IDE such as Visual Studio Code
Browser for testing	Modern browsers (Chrome, Edge, Firefox)
Storage	Browser LocalStorage (no database)
External dependencies	None (the site works offline after opening index.html)

CHAPTER

3 PROJECT DESIGN AND MODULES

3.1 System Architecture

RT Gaming Studio is a client-side application. All processing takes place in the user's browser, and there is no server component. The architecture has three layers:

- **Presentation layer (index.html + style.css)** – structure, layout, theme and animations.
- **Application layer (script.js – site logic)** – navigation, search and filtering, game window management, statistics and storage.
- **Game layer (script.js – game modules)** – one self-contained function per game, each returning a controller object with stop() and, where relevant, pause().

File	Responsibility
index.html	Page structure: loader, header, hero, game library, game of the day, leaderboard, about, footer and the shared game modal.
style.css	Light theme variables, layouts, cards, game boards, animations, responsive rules and reduced-motion support.
script.js	Storage helpers, game registry, six game modules, game window control and page initialisation.
Rishabh_Badge.png	Developer badge image displayed in the footer.

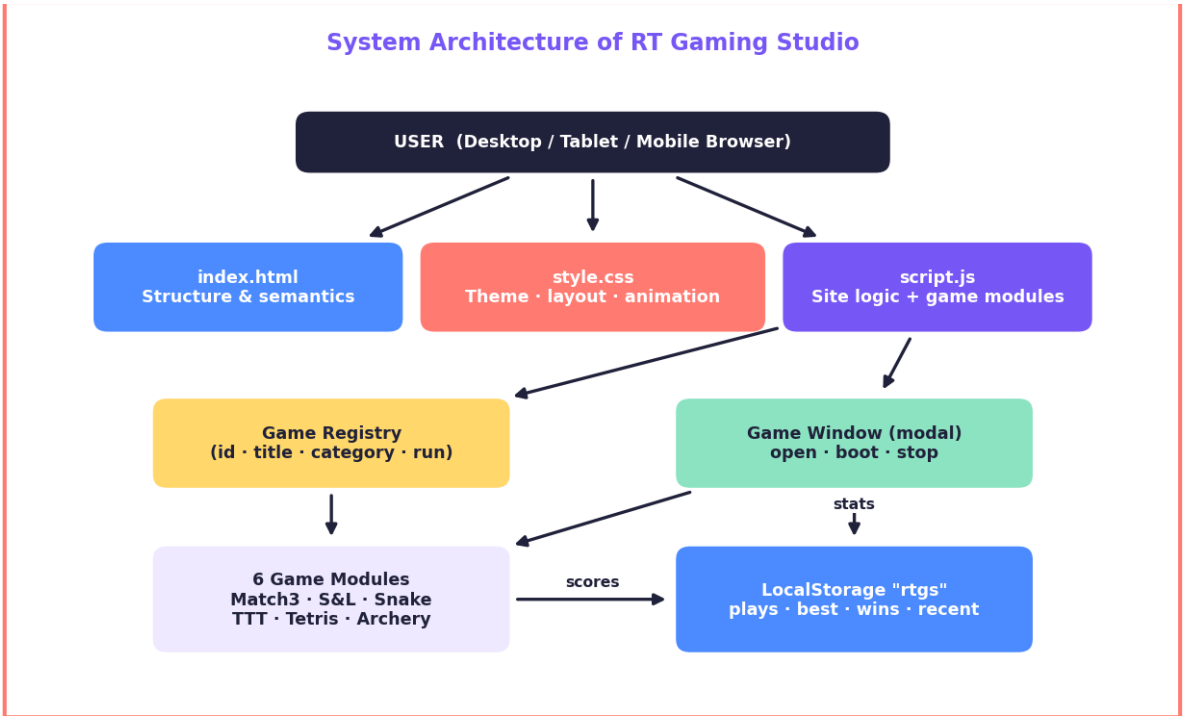


Figure 3.1: System architecture of RT Gaming Studio

Game registry. All games are described in one array. Each entry stores an id, title, category, difficulty, icon, description, instructions and a run function. The library cards, filters, search, random-game button and Game of the Day are all generated from this single array, so adding a new game only requires adding one entry and one game function.

Game lifecycle. The shared game window follows the same sequence for every game:

- openGame(id) – shows the modal, records a "play" in the statistics and calls boot().
- boot() – stops any running game, clears the game area and calls the game's run function with a small API (score and win callbacks).
- The game builds its own interface inside the game area and starts its loop or event handlers.
- On Restart, boot() is called again, giving a completely fresh state.
- On close (Close button, Back button, Escape key or backdrop click), stop() removes timers, animation frames and keyboard listeners, so nothing keeps running in the background.

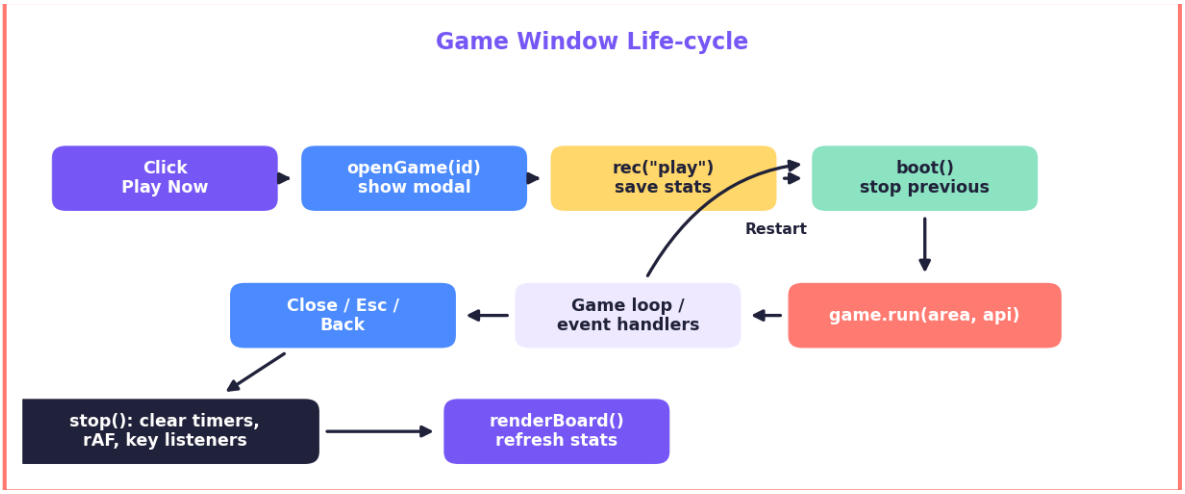


Figure 3.2: Game window life-cycle

3.2 Website Flow and Navigation

The user lands on the animated loading screen, which fades out and reveals the homepage. From the sticky header the user can move to any section using smooth scrolling. The active menu item changes automatically as the page is scrolled. On small screens the menu collapses behind a toggle button.

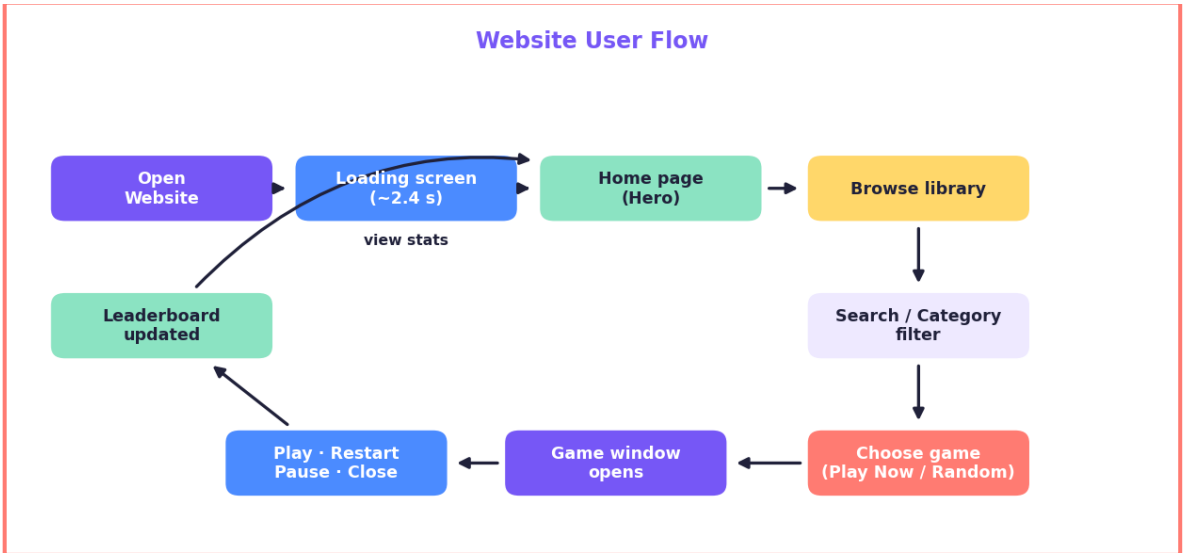


Figure 3.3: Website user flow

Menu / Action	Destination / Behaviour
Home	Hero section with Explore Games and Play Random Game buttons
Games	Game library with search and game cards
Categories	Category filter chips (All Games, Puzzle, Board Games, Arcade, Strategy, Casual)
Leaderboard	Local statistics and best scores

Menu / Action	Destination / Behaviour
About	Brand description and feature cards
Search box	Filters games by title, description and category while typing
Play Now (header)	Scrolls to the game library
Play Random Game	Opens a randomly chosen game from the registry

3.3 Loading Screen and Homepage

Loading screen. When the website opens, a full-screen lavender-to-white screen appears. The "RT" logo scales into view and glows, "RT GAMING STUDIO" fades in, a progress bar fills and the text "Preparing Your Playground..." is shown while soft shapes float in the background. After about 2.4 seconds the screen fades away. With reduced-motion enabled, the animations are shortened.

Homepage. The hero section contains the badge "WELCOME TO YOUR PLAYGROUND", the heading "One Studio. Endless Fun.", a supporting paragraph and two buttons: Explore Games and Play Random Game. A line below shows the number of games, and on the right a set of floating game-themed symbols (controller, dice, cards, puzzle piece, star and target) creates a playful illustration.

A **Game of the Day** section highlights one game chosen from the date, with its illustration, description, a Play Now button and a random-game button. The About section lists four feature cards: Instant Play, Game Categories, Mobile Friendly and Save Best Scores.

3.4 Game Library and Categories

The library is titled "Explore Our Games". Cards are created dynamically from the registry. Every card contains an illustration, title, short description, category and difficulty tags and a Play Now button. Category chips and the search box work together: a game is displayed only if it matches both the selected category and the search text. A counter shows how many games are currently visible, and a friendly empty state with a "Clear search" button appears when nothing matches.

No.	Game	Category	Difficulty
1	Candy Match	Puzzle	Easy
2	Snakes & Ladders	Board Games	Easy

No.	Game	Category	Difficulty
3	SnakeBite	Arcade	Medium
4	Tic-Tac-Toe	Strategy	Easy–Hard
5	Block Stack	Puzzle	Medium
6	Archery	Casual	Medium

3.5 Description of the Games

3.5.1 Candy Match (Match-3 Puzzle)

An 8 × 8 board is filled randomly with six types of colourful fruit-candy pieces. The player selects a piece and then an adjacent piece to swap them. If the swap does not create a row or column of three or more identical pieces, it is automatically undone. Valid matches are removed, pieces above fall down, new pieces fill the empty spaces and cascades are detected repeatedly. Points increase with the size of the match and the cascade multiplier. The player must reach 1000 points within 20 moves to win. The board is generated so that it never starts with an existing match and always has at least one possible move; if no moves remain after a cascade, the board is reshuffled.

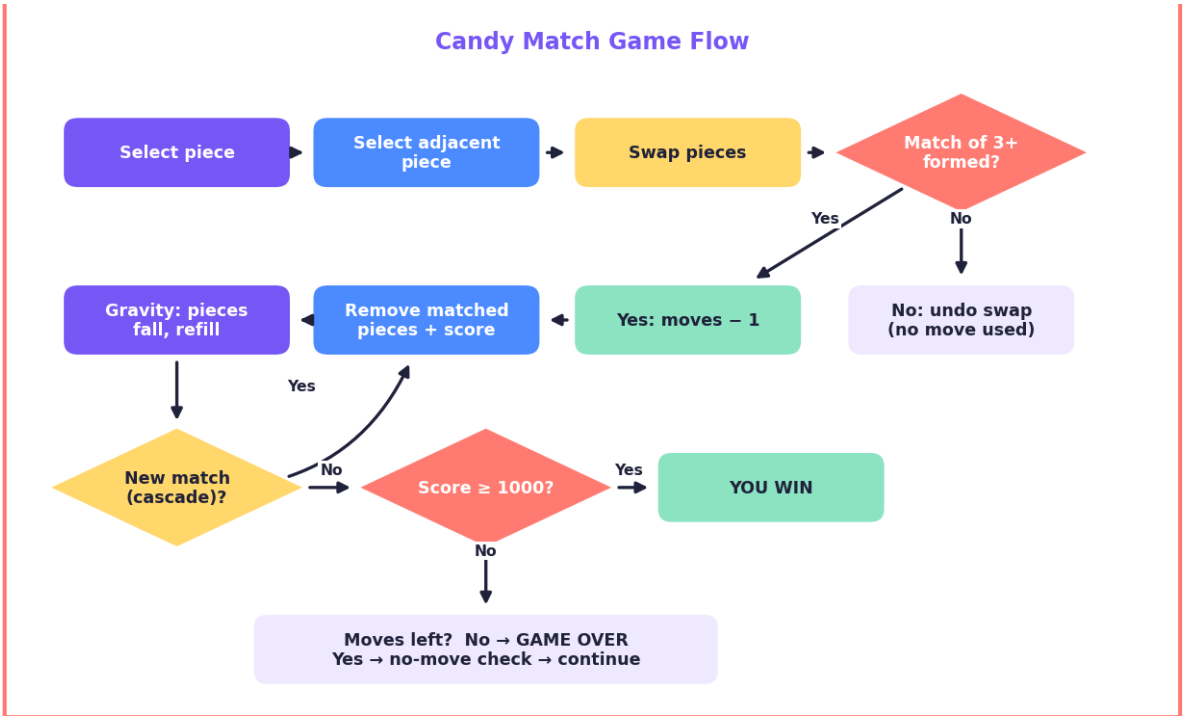


Figure 3.4: Candy Match game flow

3.5.2 Snakes and Ladders

A 10 × 10 board numbered 1 to 100 with alternating row direction (zig-zag). The board contains eight ladders and eight snakes, shown with icons and their destination numbers. Two local players (red and blue) take turns rolling an animated dice; the token moves one square at a time. Landing on a ladder climbs up and landing on a snake slides down. The overshoot rule is that the player must roll the exact number to land on square 100; a larger roll leaves the token where it is. The first player to reach 100 wins.

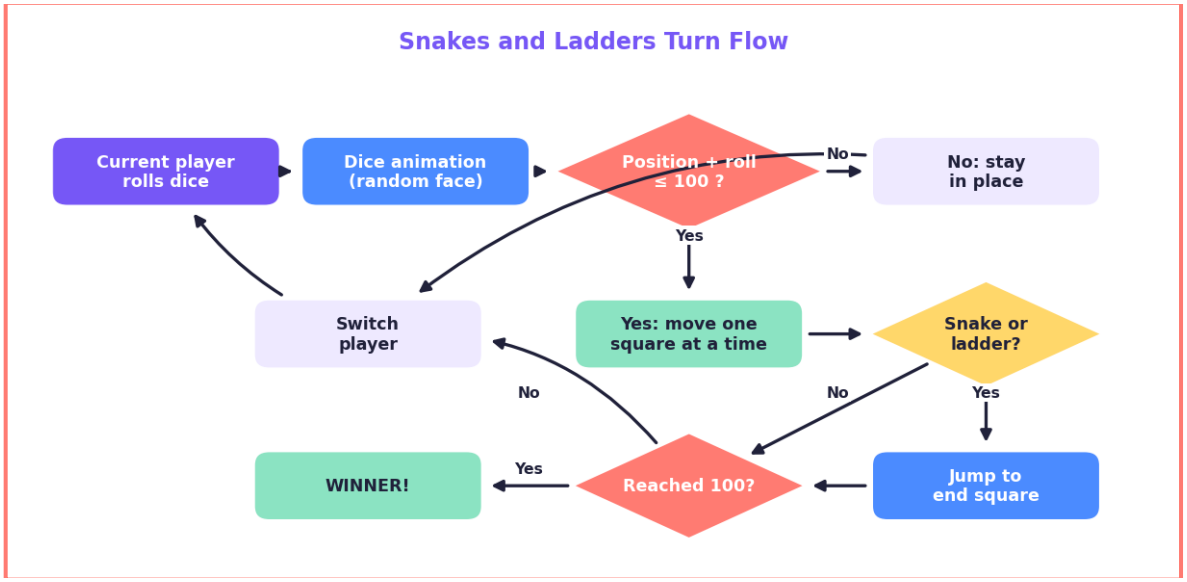


Figure 3.5: Snakes and Ladders turn flow

3.5.3 SnakeBite

A classic snake game on a 20 × 20 grid drawn on a canvas. The snake moves automatically every 110 ms. Eating food increases the length and adds 10 points. The game ends when the snake hits a wall or its own body. Direction can be changed with arrow keys, W-A-S-D or an on-screen direction pad, and direct reversal into the snake's own body is prevented. A Pause/Resume button is available and the best score is saved.

3.5.4 Tic-Tac-Toe

A 3 × 3 game with two modes: two players, or player versus computer. In computer mode there are two difficulty levels. Easy mode selects a random free square. Hard mode uses the minimax algorithm to evaluate every possible continuation and therefore never loses. The winning line is highlighted, the current turn is shown, and a scoreboard counts X wins, O wins and draws. Buttons are provided to reset scores and to change mode.

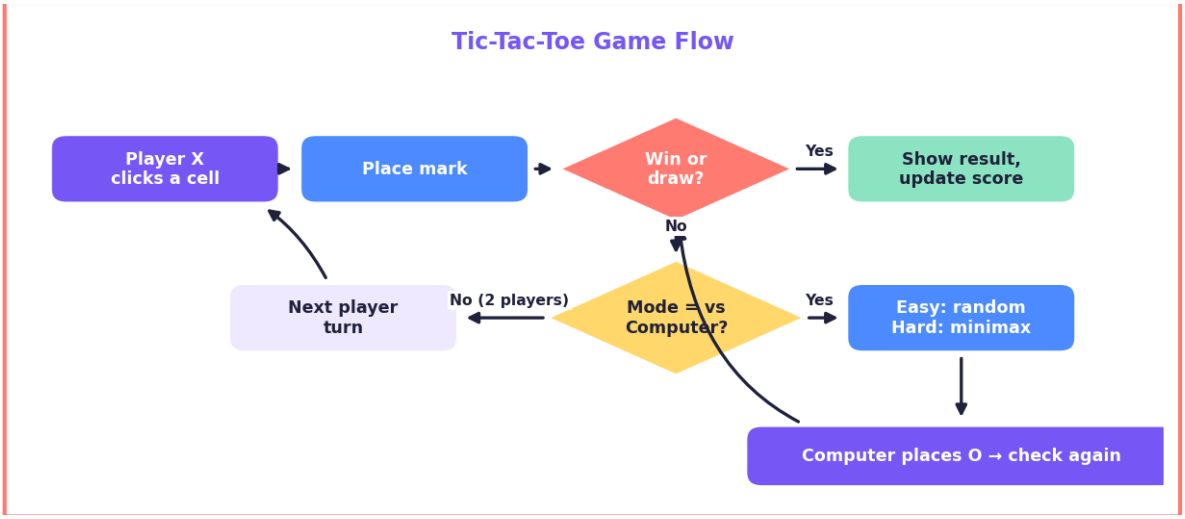


Figure 3.6: Tic-Tac-Toe game flow

3.5.5 Block Stack (Tetris-style)

A 10 × 20 board where the seven standard tetromino shapes (I, O, T, S, Z, J, L) fall one after another. The player can move left or right, rotate, soft drop and hard drop; collision detection prevents pieces from passing through walls or locked blocks. Completed lines are cleared, and the score, level and line counters are updated. A level increases every 10 lines and the falling speed increases with the level. A preview of the next piece is shown, the game can be paused with the P key, and the game ends when new pieces can no longer be placed.

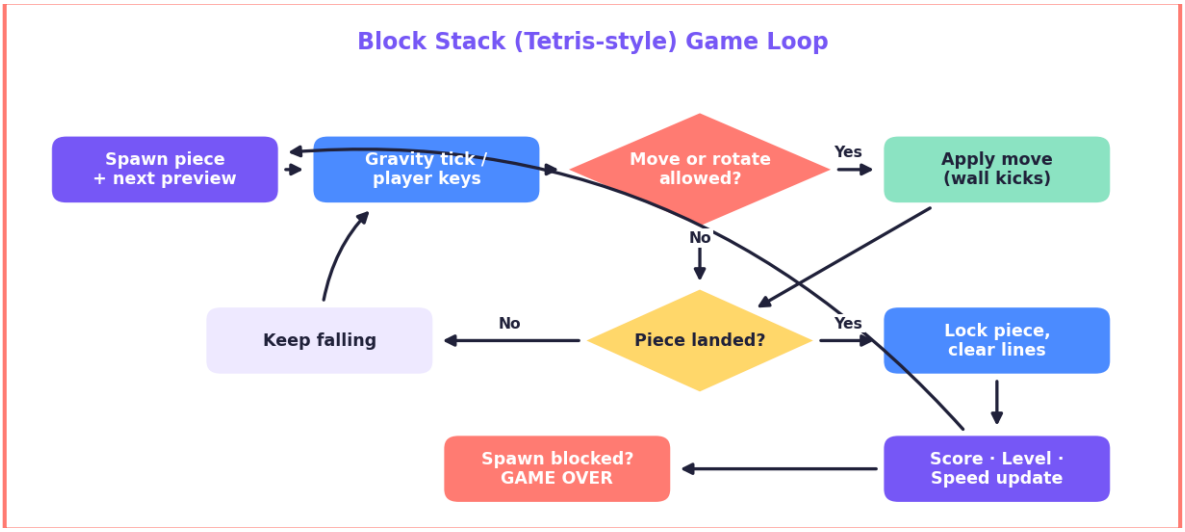


Figure 3.7: Block Stack game loop

3.5.6 Archery

An outdoor range with a ten-ring target. The player aims using the mouse or touch, and a crosshair shows the aim point. Wind strength and direction are displayed for every

arrow and push the arrow sideways, while a small gravity drop lowers the impact point. After pressing Shoot (or Space) the arrow flies along a curved path to the calculated impact point. The score depends on the distance of that point from the bullseye, from 10 points for the centre to 1 point for the outer ring. Ten arrows form a round, after which the total and the best score are displayed.

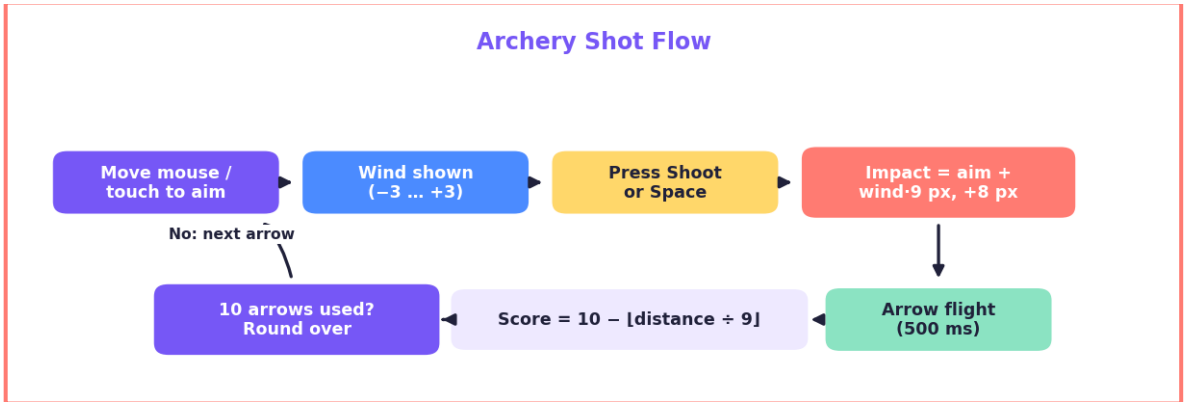


Figure 3.8: Archery shot flow

3.6 Scoreboard and Statistics

The Leaderboard section is clearly labelled as local to the current browser and device. It does not claim any global competition. It consists of three cards:

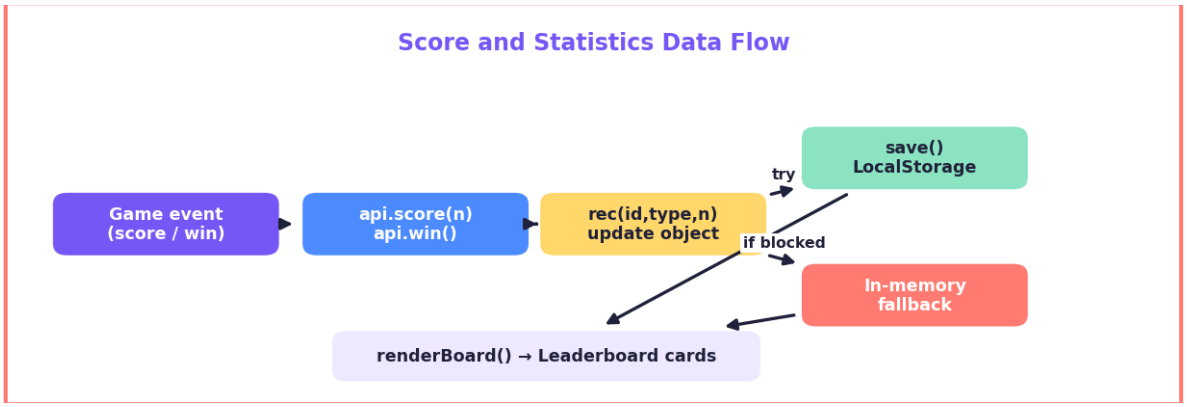


Figure 3.9: Score and statistics data flow

- **Overview** – total games played, total wins and the most-played game.
- **Best scores** – the highest score for each game with a progress bar relative to the highest score overall.
- **Recently played** – the last five games opened.

A “Reset local statistics” button clears all stored data after the user confirms the action.

CHAPTER

4 SYSTEM IMPLEMENTATION

4.1 Project Folder Structure

The complete website is delivered as a single folder. Opening index.html in a browser starts the application.

```
RT-GAMING-STUDIO/
├── index.html          (page structure)
├── style.css           (all styling and animations)
├── script.js           (site logic and all games)
└── Rishabh_Badge.png   (footer developer badge)
```

4.2 Frontend Implementation

The user interface follows a light theme only. The colour palette is defined once as CSS variables and reused across the site.

Purpose	Colour	Code
Page background	Soft blue-white	#F8FAFF
Cards	White	#FFFFFF
Primary	Purple	#7657F6
Soft primary	Light lavender	#EEE9FF
Secondary	Electric blue	#4B8BFF
Accent	Coral / Yellow / Mint	#FF7B72 / #FFD76A / #8BE3C2
Main text	Dark navy	#20213A
Borders	Light grey	#E8E9F2

- **Header:** sticky bar with the logo, navigation, search box, decorative avatar and Play Now button; it collapses into a menu button below 820 px.
- **Game cards:** grid with auto-fill columns that becomes a two-column layout on phones.
- **Game window:** a fixed overlay with a white panel, title, instructions, game area and control buttons. It scrolls on small screens.
- **Boards:** CSS grid for Candy Match, Snakes and Ladders and Tic-Tac-Toe; canvas elements that scale down with max-width for the other games.

- **Footer:** four-column layout with brand description, quick links, categories and the developer badge, plus a bottom bar showing the dynamic copyright year and "Designed & Developed by RT1504".
- **Accessibility:** visible focus outlines, ARIA labels on icon buttons and boards, live regions for status messages and reduced-motion support.

4.3 JavaScript Game Logic

script.js is organised into clearly commented parts: storage helpers, one function for each game, the game registry, library rendering, game window control and page initialisation. Important algorithms are summarised below.

Game	Key logic
Candy Match	find() scans rows and columns and collects every cell that belongs to a run of three or more using a Set; settle() repeatedly removes matches, applies gravity column by column, refills with random pieces and repeats until no match remains. hasMove() tries every neighbouring swap to make sure the board is playable.
Snakes & Ladders	A table of jumps (start → end) is checked after every move; the board number is computed from row and column to produce the zig-zag layout; async/await is used for the dice and step animation.
SnakeBite	The snake is an array of cells; each tick adds a new head and removes the tail unless food was eaten; collision tests compare the new head with the walls and the body.
Tic-Tac-Toe	Eight winning lines are tested after each move; the recursive minimax function scores terminal positions as +1 (O wins), −1 (X wins) or 0 (draw) and the computer chooses the move with the best value.
Block Stack	Each tetromino is a matrix; rotation transposes and reverses the matrix with small wall-kick offsets; hit() tests walls, floor and locked blocks; full rows are filtered out and empty rows are added at the top.
Archery	Impact point = aim point + wind × 9 pixels horizontally and + 8 pixels downwards. Score = 10 − floor(distance ÷ 9), and 0 beyond the outer ring. The same impact point is used for drawing and scoring, so the display and the score always match.

Resource cleanup. Every game returns a controller object with a stop() method. It clears intervals, cancels animation frames and removes keyboard listeners; asynchronous games use a "dead" flag so that delayed callbacks do nothing after the window is closed. This prevents background execution and duplicate event listeners.

4.4 Game Controls and Score Management

Game	Mouse / Touch	Keyboard
Candy Match	Tap a piece, then an adjacent piece	—
Snakes & Ladders	Roll dice button	—
SnakeBite	On-screen direction pad	Arrow keys, W A S D
Tic-Tac-Toe	Tap a square	—
Block Stack	On-screen buttons: left, right, rotate, soft drop, hard drop	← → move, ↑ rotate, ↓ soft drop, Space hard drop, P pause
Archery	Move/touch to aim, Shoot button	Space to shoot

Score management is centralised. Each game receives a small API with two functions, score(n) and win(). The score function saves a new best score only if it is higher than the stored best; the win function increases the total number of wins. The play count and recent list are updated when a game is opened. All statistics are stored in one JSON object:

Field	Type	Meaning
plays	Object	Number of times each game was opened
best	Object	Best score of each game
wins	Number	Total wins across all games
recent	Array	Last five games played

CHAPTER

5 SOFTWARE TESTING

Testing verifies that the website and the games behave as expected. Test cases were designed for the functional features of the website, for the rules of every game and for responsive behaviour. In the Result column, “Pass” marks cases that were verified by an automated browser script (Playwright with Chromium) run against the project files; a dash (—) marks cases that still have to be verified manually.

5.1 Functional Testing

Functional testing checks the website features such as the loader, navigation, search, filtering, game opening and statistics.

ID	Test Case	Expected Result	Result
FT-01	Open the website	Loading screen appears and fades out after about 2–3 seconds, revealing the homepage	Pass
FT-02	Click each navigation link	Page scrolls smoothly to the correct section and the active link changes	Pass
FT-03	Type a game name in search	Only matching games are shown and the counter updates	Pass
FT-04	Select a category chip	Only games of that category are displayed	Pass
FT-05	Combine search text and category	Both filters are applied together	Pass
FT-06	Search for a non-existing game	Empty-state message and Clear search button are shown	Pass
FT-07	Click Play Random Game	A game from the library opens in the game window	Pass
FT-08	Click Play Now on every card	The matching game opens with its title and instructions	Pass
FT-09	Press Escape / Close / Back	Game window closes and the game stops	Pass
FT-10	Play a game and open Leaderboard	Games played, best score and recent list are updated	Pass
FT-11	Click Reset local statistics	After confirmation all statistics are cleared	Pass
FT-12	Block LocalStorage in the browser	Website still works and keeps data in memory without errors	Pass

5.2 Game Testing

Game testing verifies the rules, controls, scoring and restart behaviour of each game.

ID	Test Case	Expected Result	Result
GT-01	Candy Match: swap without a match	Swap is undone and no move is used	—
GT-02	Candy Match: valid swap	Matches are removed, pieces fall, new pieces appear and score increases	—
GT-03	Candy Match: reach 1000 points / use 20 moves	Win or game-over message is displayed	—
GT-04	Snakes & Ladders: roll dice	Dice animation runs and the token moves the rolled number of squares	—
GT-05	Snakes & Ladders: land on snake / ladder	Token slides down or climbs to the correct square	—
GT-06	Snakes & Ladders: roll more than needed near 100	Token stays; turn passes to the other player	—
GT-07	SnakeBite: eat food	Snake grows by one and score increases by 10	—
GT-08	SnakeBite: press the opposite direction	Snake does not reverse into itself	—
GT-09	SnakeBite: hit wall or body	Game-over message appears and the loop stops	—
GT-10	SnakeBite: Pause / Resume	Movement stops and continues correctly	—
GT-11	Tic-Tac-Toe: three in a row	Winning line is highlighted and score is updated	**Pass**
GT-12	Tic-Tac-Toe: hard mode	Computer never loses	—
GT-13	Tic-Tac-Toe: full board without winner	Draw is declared	—
GT-14	Block Stack: move and rotate at walls	Piece never leaves the board or overlaps locked blocks	—
GT-15	Block Stack: complete a row	Row is cleared and score, lines and level update	—
GT-16	Block Stack: Pause with P	Pieces stop falling until resumed	—
GT-17	Archery: shoot with wind	Arrow lands at the displayed impact point and score matches the ring	**Pass**
GT-18	Archery: fire ten arrows	Round ends and total and best score are shown	—
GT-19	Restart every game	A fresh game starts with reset score and state	**Pass**

ID	Test Case	Expected Result	Result
GT-20	Close a game and wait	No timers, sounds or key handling continue in the background	Pass

5.3 Responsive UI Testing

Responsive testing confirms that the layout and controls remain usable on different screen sizes and accessibility settings.

ID	Test Case	Expected Result	Result
UI-01	Desktop width (≥ 1200 px)	Full navigation and multi-column game grid	—
UI-02	Tablet width (about 768 px)	Menu button appears and grid adjusts	—
UI-03	Mobile width (about 375 px)	Two-column cards, no horizontal page scrolling	Pass
UI-04	Touch input on phone	Boards, direction pad and buttons respond to touch	—
UI-05	Game window on small screen	Panel scrolls and the board fits the screen	—
UI-06	Keyboard-only navigation	Focus outline is visible and buttons are reachable	—
UI-07	Reduced-motion setting enabled	Animations are minimised	Pass
UI-08	Light theme on all screens	No dark theme appears anywhere	—

5.4 Test Case Summary

Test Category	Designed	Verified Pass	Pending Manual
Functional Testing	12	12	0
Game Testing	20	4	16
Responsive UI Testing	8	2	6
Total	**40**	**18**	**22**

AUTOMATED CHECK RESULT

No console or page errors were reported during the automated run, which opened all six games, played them, searched and filtered the library, used the reset button and ran the site with LocalStorage blocked.

CHAPTER

6 SCREENSHOTS AND WALKTHROUGH

This chapter presents the working website step by step. All screenshots in this chapter were captured from the running project in a Chromium browser. The same project is available online at the live link given on the last page.

6.1 Loading Screen and Home Page

The first screen shows the animated RT logo, the brand name and the loading bar. After it fades, the homepage appears with the hero section, floating game symbols and call-to-action buttons.



Figure 6.1: Loading screen

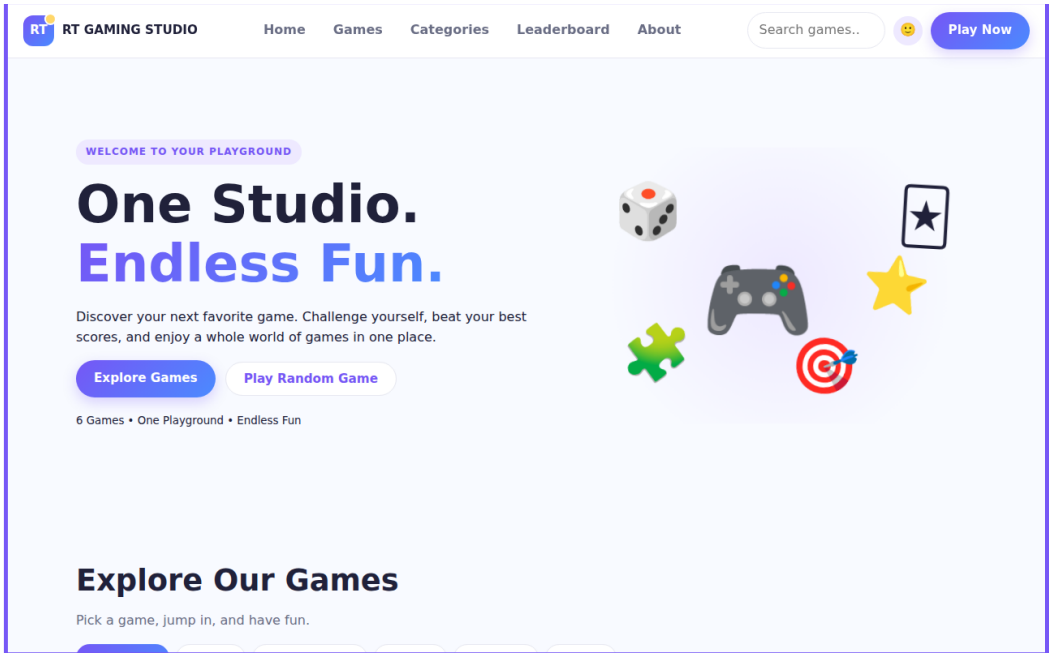


Figure 6.2: Home page (hero section)

6.2 Game Library and Search

The game library displays the game cards with category chips and the search box. Typing in the search box or choosing a category filters the cards immediately and shows the current count.

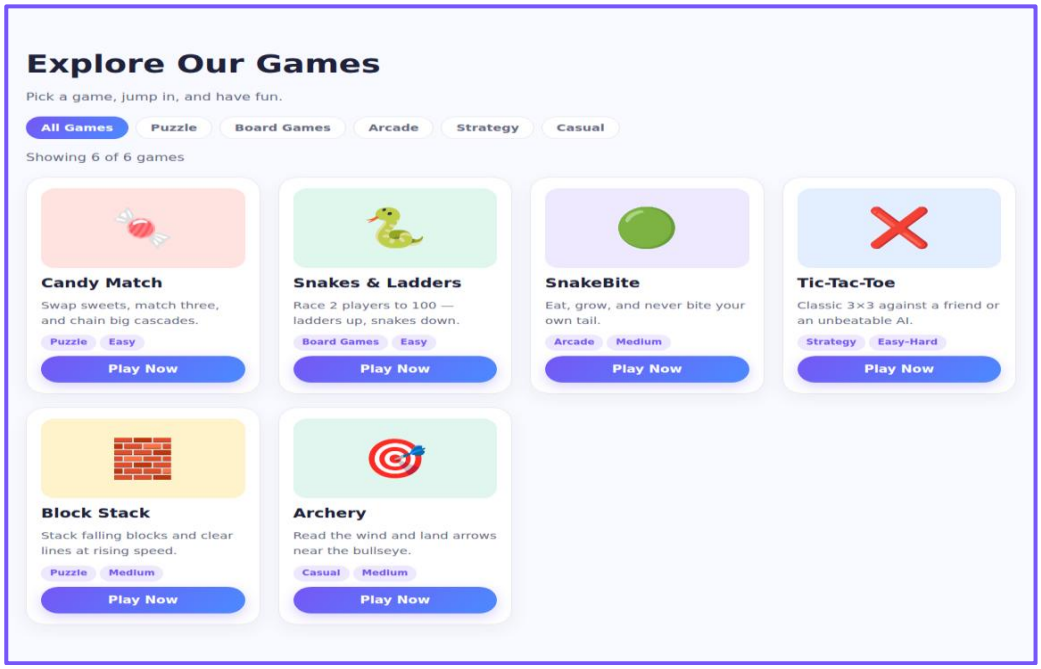


Figure 6.3: Game library with category filters

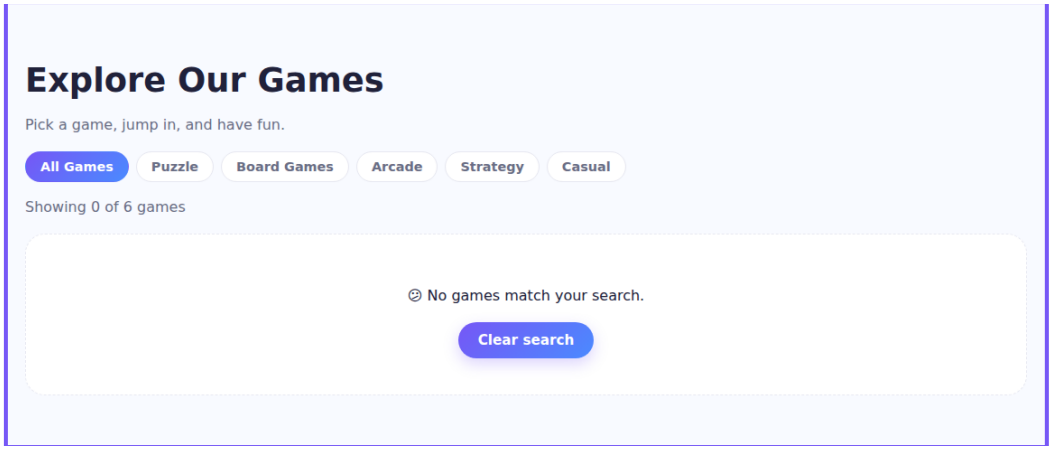


Figure 6.4: Search result and empty state

6.3 Gameplay Screenshots

Each game opens in the shared game window with its title, instructions and control buttons.

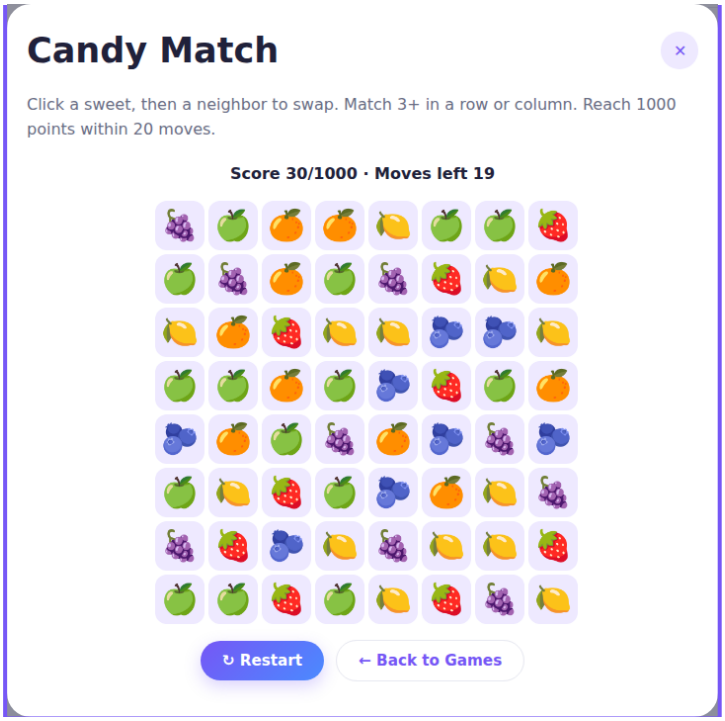


Figure 6.5: Candy Match gameplay

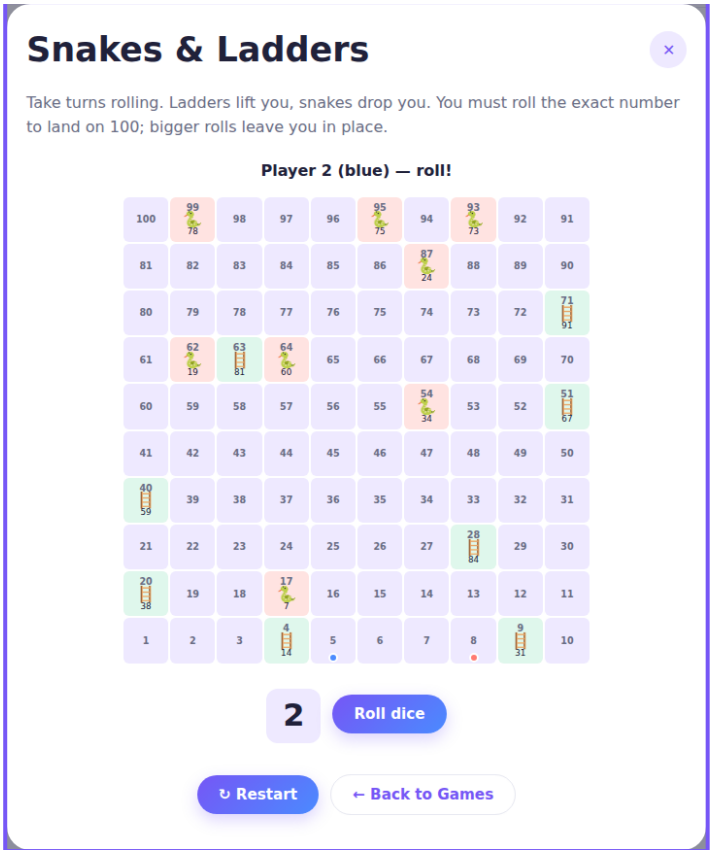


Figure 6.6: Snakes and Ladders gameplay

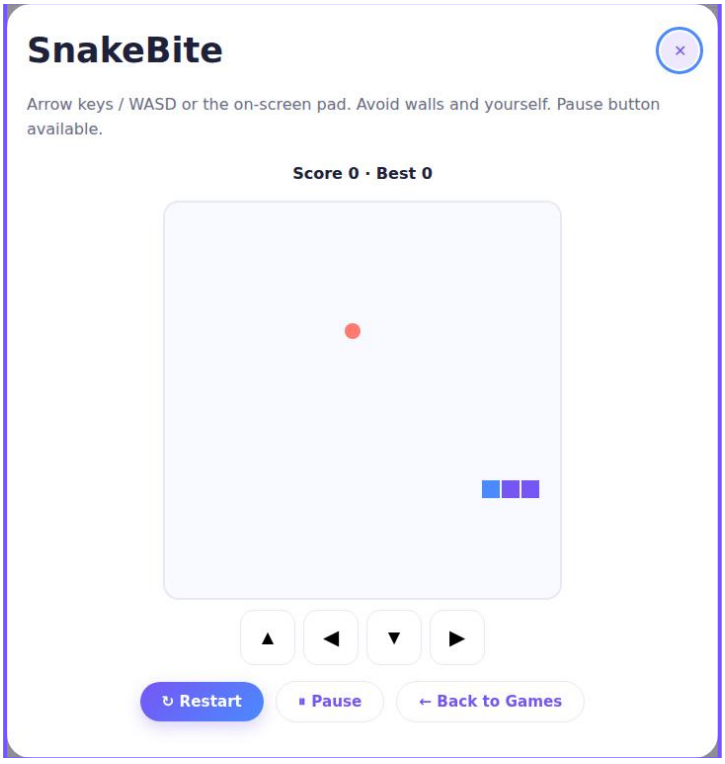


Figure 6.7: SnakeBite gameplay



Figure 6.8: Tic-Tac-Toe gameplay

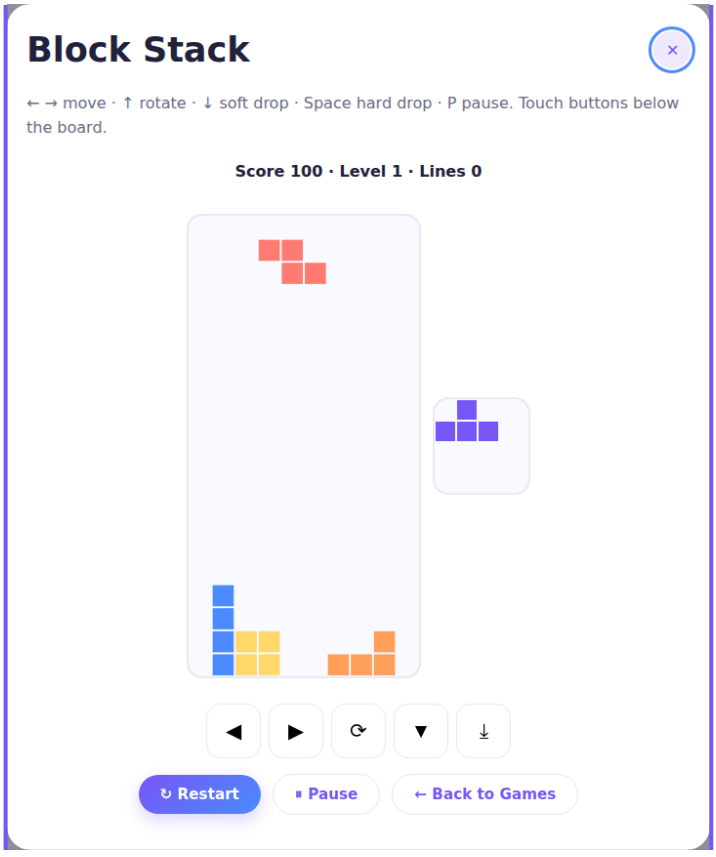


Figure 6.9: Block Stack gameplay

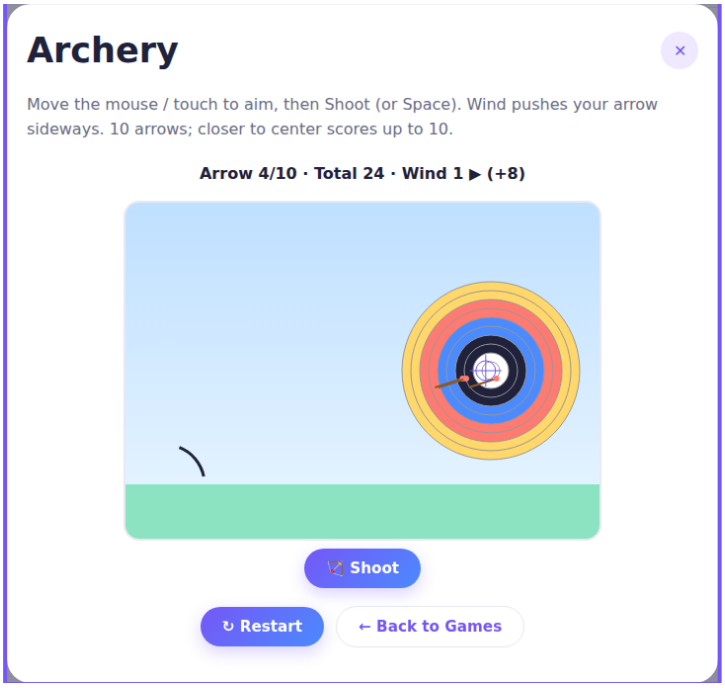


Figure 6.10: Archery gameplay

6.4 Scoreboard and Statistics

The Leaderboard section shows the local overview, best scores with progress bars and recently played games. The footer and the About section complete the page.

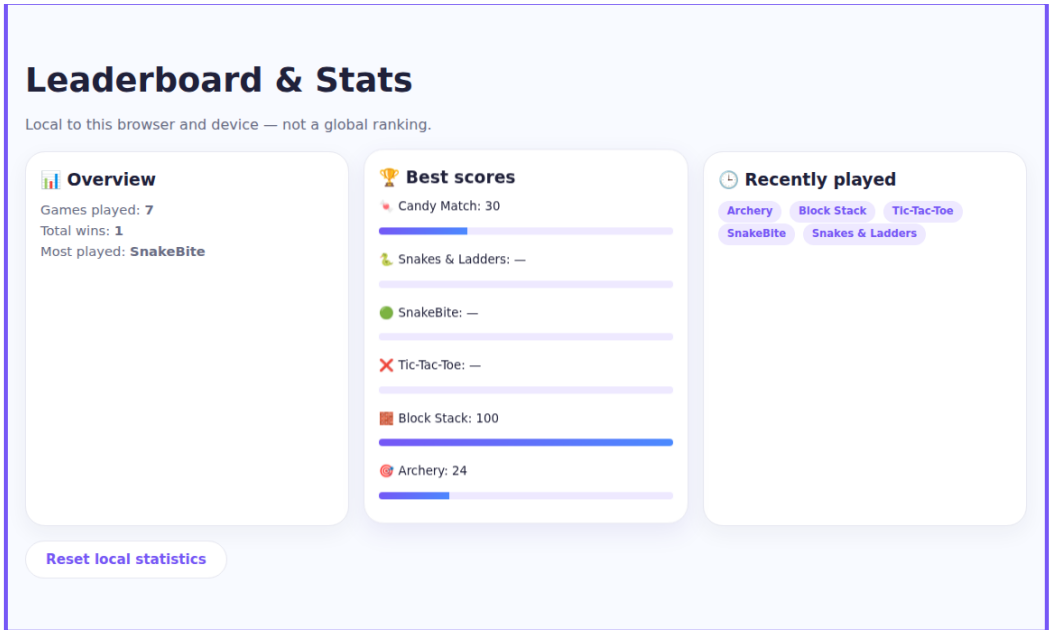


Figure 6.11: Leaderboard and statistics

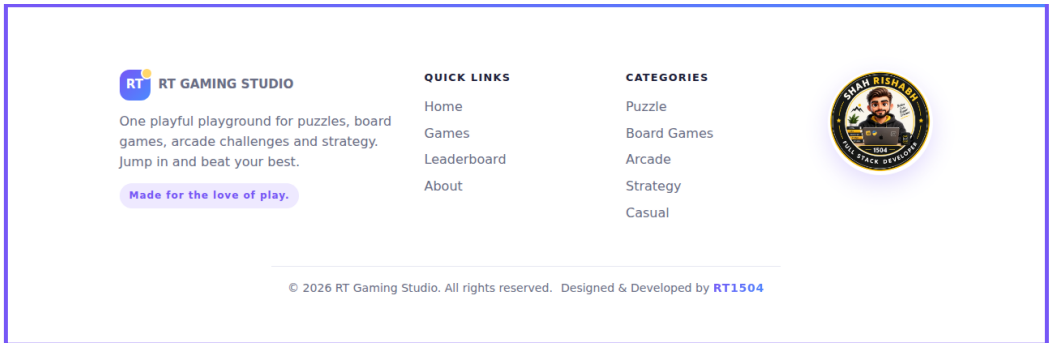


Figure 6.12: Footer with developer badge

▸ Mobile view

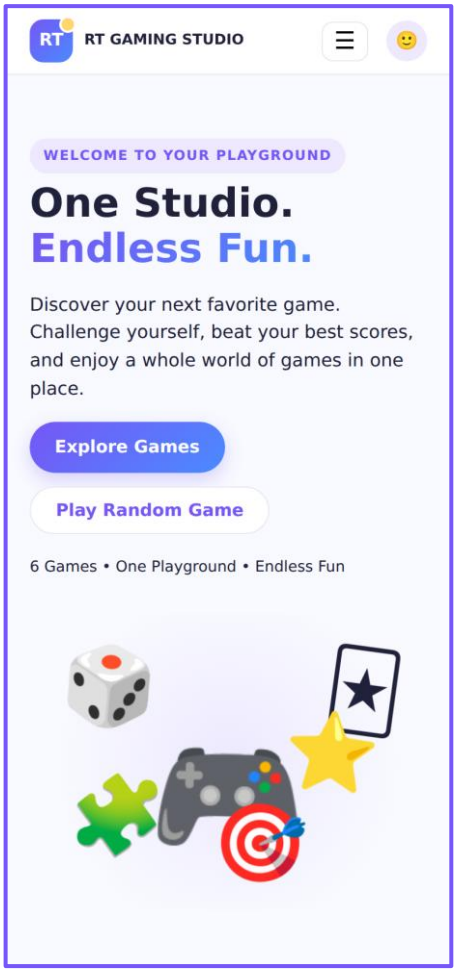


Figure 6.13(a): Mobile home screen

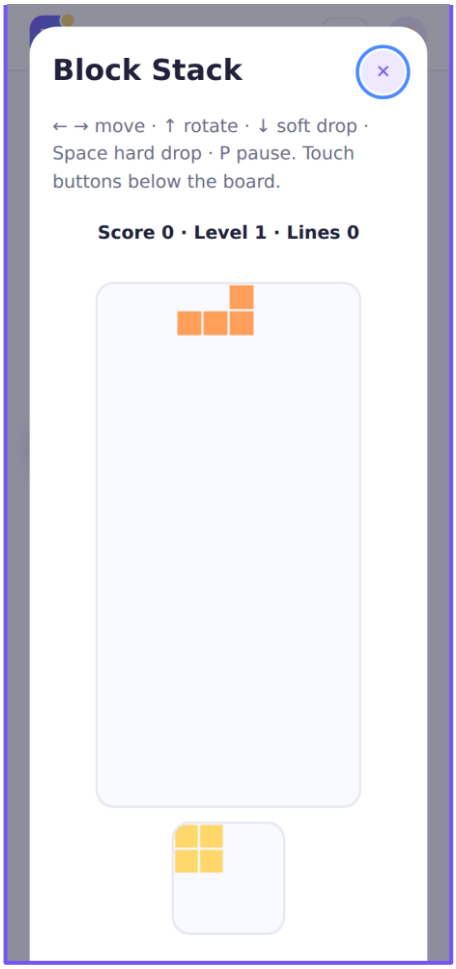


Figure 6.13(b): Block Stack with touch controls

CHAPTER

7 LEARNING OUTCOMES

7.1 Skills and Knowledge Gained

Area	Learning
HTML5	Semantic structure, canvas element, accessibility attributes and form controls.
CSS3	Flexbox, Grid, variables, animations, media queries and responsive design.
JavaScript	ES6+ syntax, DOM handling, events, timers, async/await, closures and modular code organisation.
Algorithms	Grid matching, gravity and refill, recursion and minimax, collision detection, matrix rotation and geometry-based scoring.
Browser APIs	LocalStorage with safe fallback, Canvas 2D, requestAnimationFrame and pointer events.
Design	Creating a brand identity, consistent light theme and user-friendly layout.
Software practice	Planning, modular design, testing, documentation and clean-up of resources.

7.2 Challenges Faced and Solutions

Challenge	Solution
Games continuing to run after closing the window	Every game returns a stop() method; intervals, animation frames and key listeners are cleared and asynchronous code checks a “dead” flag.
Candy Match boards starting with matches or no possible move	The board is regenerated until it has no matches and at least one valid swap; reshuffling is done after cascades if needed.
Cascades and animations overlapping with user clicks	A busy flag blocks input while the board is settling.
Snake reversing into itself with quick key presses	Direction changes are compared with the direction of the last move and opposite directions are ignored.
Rotated Tetris pieces colliding with walls	Small left/right offsets (wall kicks) are tried before rejecting a rotation.
Archery display not matching the score	The impact point is calculated once and used for both drawing and scoring.
LocalStorage may be disabled	All access is wrapped in try/catch with an in-memory fallback.

Challenge	Solution
Different screen sizes	Relative units, grid layouts, scalable canvases, breakpoints and touch buttons.
CSS class name clash (“c” used for both a game piece and a button row) found during screenshot testing	The game piece class was renamed so that styles no longer leak into other elements.
Hidden Pause button still visible on games without pause	A global [hidden] rule with higher priority was added so the HTML hidden attribute always works.
Invalid CSS font shorthand made some text and X/O marks too small	The shorthand was replaced with separate font-weight, font-size and font-family properties.

7.3 Limitations and Future Scope

Limitations

- The current version contains six games.
- Statistics are saved only in one browser on one device and cannot be shared.
- Games are for local play; there is no online multiplayer.
- Sound effects are not yet available.

Future scope

- Adding more games such as Chess, Ludo and an UNO-style card game.
- Adding optional sound effects with an on/off switch.
- Adding an installable Progressive Web App version for offline use.
- Adding difficulty levels and achievements for more games.

CHAPTER

8 CONCLUSION

8.1 Conclusion

RT Gaming Studio successfully demonstrates that a complete, attractive and interactive gaming platform can be built with only HTML5, CSS3 and Vanilla JavaScript. The project combines a polished light-themed interface with six genuinely playable games, a shared game window, local statistics and responsive behaviour for desktop and mobile devices.

The use of a game registry keeps the code organised and extensible, while careful cleanup of timers and listeners keeps the website stable. Working on this project improved my understanding of front-end development, game logic, algorithm design, user-interface design and testing. The final result is a practical and enjoyable web application that can be improved further with more games and features.

8.2 Future Enhancements

- Implement Chess with full legal-move validation, castling, en passant, promotion, check and checkmate.
- Implement Ludo for two to four local players.
- Implement an UNO-style card game against computer opponents.
- Add sound effects using the Web Audio API with a mute option.
- Add a player profile (name and avatar) stored locally.
- Add daily challenges and achievement badges.
- Make the website installable as a Progressive Web App.
- Add optional online features such as a shared leaderboard with a backend service.

CHAPTER 9

REFERENCES AND GLOSSARY

9.1 References

- MDN Web Docs – HTML, CSS and JavaScript documentation. https://developer.mozilla.org
- WHATWG – HTML Living Standard. https://html.spec.whatwg.org
- W3C – CSS specifications. https://www.w3.org/Style/CSS/
- ECMA International – ECMAScript Language Specification. https://tc39.es/ecma262/
- MDN Web Docs – Canvas API and Web Storage API. https://developer.mozilla.org
- W3C – Web Content Accessibility Guidelines (WCAG) 2.1. https://www.w3.org/TR/WCAG21/
- Russell, S. and Norvig, P. – Artificial Intelligence: A Modern Approach (minimax search).

9.2 Glossary

Term	Meaning
HTML5	The latest version of HyperText Markup Language used to structure web pages.
CSS3	Cascading Style Sheets, used for the design and layout of web pages.
JavaScript	A programming language that adds behaviour and interactivity to web pages.
Vanilla JavaScript	Plain JavaScript without frameworks or libraries.
DOM	Document Object Model – the browser's tree representation of the page.
Canvas	An HTML element for drawing 2D graphics with JavaScript.
LocalStorage	Browser storage for key–value data that stays after the page is closed.
Game registry	A central list that describes all games and how to start them.
Minimax	A decision algorithm that assumes both players play optimally.
Cascade	Chain reaction in a match-3 game when falling pieces create new matches.
Tetromino	A shape made of four connected squares used in Tetris-style games.
Responsive design	Layout that adapts to different screen sizes.
Reduced motion	A user setting that asks websites to minimise animation.
ARIA	Accessible Rich Internet Applications – attributes that improve accessibility.

CONNECT WITH ME

Thank you for reading this report. Use the buttons below to explore the project, view the source code or get in touch.

VIEW PROJECT REPOSITORY	**OPEN LIVE PROJECT**
GITHUB PROFILE	**LINKEDIN PROFILE**



SHAH RISHABH ALPESHBHAI

Full Stack Developer · RT1504

"Better Code, Bigger Dreams."